

239. 滑动窗口最大值

239. 滑动窗口最大值

1.问题

滑动窗口最大值

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard (49.43%)	3111	0	-	-	0

▼ Tags

▼ Companies

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 *滑动窗口中的最大值*。

示例 1:

输入: nums = [1,3,-1,-3,5,3,6,7], k = 3

输出: [3,3,5,5,6,7]

解释:

滑动窗口的位置

最大值

[1 3 -1] -3 5 3 6 73

1 [3 -1 -3] 5 3 6 73

1 3 [-1 -3 5] 3 6 7 5

1 3 -1 [-3 5 3] 6 7 5

1 3 -1 -3 [5 3 6] 7 6

1 3 -1 -3 5 [3 6 7] 7

示例 2:

输入: nums = [1], k = 1

输出: [1]

提示:

- `1 <= nums.length <= 105`
- `104 <= nums[i] <= 104`
- `1 <= k <= nums.length`

2.解析

239. 滑动窗口最大值 - 详细解析

问题分析

这道题要求在大小为k的滑动窗口中找出每个窗口的最大值。关键挑战是如何在O(n)时间内高效地维护和更新最大值。

解题思路

可以使用**双端队列(Deque)**来解决，保持队列中的元素索引对应的值是单调递减的。这样队首元素就是当前窗口的最大值。

具体步骤：

1. 维护一个双端队列，存储元素在原数组中的索引
2. 保持队列中的元素从大到小排列(单调递减队列)
3. 当窗口滑动时，如果队首元素不在当前窗口内，则将其移除
4. 当新元素进入队列前，先从队尾移除所有小于它的元素

代码实现

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        // 特判
        if (nums == null || nums.length == 0 || k <= 0) {
            return new int[0];
        }

        int n = nums.length;
        int[] result = new int[n - k + 1]; // 结果数组大小
        int resultIndex = 0; // 结果数组的索引

        // 双端队列存储元素索引
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < nums.length; i++) {
            // 1. 移除队列中不在当前窗口内的元素索引
            // 如果队首元素(最大值)已经超出窗口范围，则移除
            while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
                deque.pollFirst();
            }

            // 2. 移除队列中所有小于当前元素的值
            // 因为这些值不可能是后续窗口的最大值
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }

            // 3. 将当前元素的索引加入队列
            deque.offerLast(i);

            // 4. 如果已经形成了完整窗口，则记录当前窗口的最大值
            if (i >= k - 1) {
                result[resultIndex++] = nums[deque.peekFirst()];
            }
        }

        return result;
    }
}
```

示例详解

以输入 `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` 为例，执行过程：

1. `i=0` : 处理元素1

- 队列为空，加入索引0
 - 队列：[0]
2. `i=1` : 处理元素3
- $3 > \text{nums}[0]=1$ ，移除队列中所有元素
 - 加入索引1
 - 队列：[1]
3. `i=2` : 处理元素-1
- $1 < \text{nums}[1]=3$ ，直接加入索引2
 - 队列：[1,2]
 - 此时窗口大小达到 $k=3$ ，记录最大值 $\text{nums}[1]=3$
 - 结果：[3]
4. `i=3` : 处理元素-3
- 移除不在窗口内的元素（无）
 - $3 < \text{nums}[2]=-1$ ，直接加入索引3
 - 队列：[1,2,3]
 - 记录最大值 $\text{nums}[1]=3$
 - 结果：[3,3]
5. `i=4` : 处理元素5
- 索引1不在当前窗口[2,3,4]内，移除
 - 5大于队列中所有元素对应的值，清空队列
 - 加入索引4
 - 队列：[4]
 - 记录最大值 $\text{nums}[4]=5$
 - 结果：[3,3,5]
6. 依此类推...

最终输出：[3,3,5,5,6,7]

复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 是数组长度，每个元素最多进出队列各一次
- 空间复杂度： $O(k)$ ，双端队列最多同时存储 k 个元素的索引

优化思路

1. 可以使用优先队列（最大堆）来解决，但时间复杂度会变为 $O(n\log k)$
2. 本解法使用双端队列是最优的时间复杂度解法
3. 如果数据规模小，可以考虑直接使用暴力解法，遍历每个窗口找最大值